Kurseinheit	Phase	Beschreibung	Theorie (Quelle) bzw. Praxis (Quelle)	Zeit Lehrer bzw. Schüler
		Leitidee: "Wie können wir Objekte mit einer Kamera entdecken, um damit einen ESP32 anzusteuern?"		
K1	Thema	Was sieht die Kamera?		
K2	Thema	Wie lernt eine Maschine?		
K3	Thema	KI-Modell trainieren		
K3	Theorie	Rolle von Edge Impulse Training Supervised Learning Overfitting	**Edge Impulse** ist eine Plattform, die den Prozess der Erstellung von Machine-Learning-Modellen **vereinfacht**, indem sie sinnvolle Standardwerte für die unzähligen Parameter auswählt, die bei der Erstellung eines ML-Modells festgelegt werden könnten. Sie bietet eine einfache Benutzeroberfläche, die es nicht nur ermöglicht, ein ML-Modell zu trainieren, sondern auch, die Daten zu inspizieren und das Modell zu testen. (...) Um ein ML-Modell für die Klassifizierung von Bildern zu **trainieren**, müssen wir es mit Bilddaten des entsprechenden Objekts **füttern**. Während des Trainingsprozesses wird das Modell mithilfe eines Konzepts geschult, das als „überwachtes Lernen“ (**Supervised Learning**) bezeichnet wird. Dies bedeutet, dass wir das Modell mit bekannten Daten trainieren und ihm – während es seine Vorhersagen „übt“ – mitteilen, ob diese korrekt sind oder nicht. (...) Ein Aspekt, den es zu berücksichtigen gilt, ist das **Overfitting**. Wenn ein Machine-Learning-Modell ein Overfitting aufweist, bedeutet dies, dass es zu stark auf deine Trainingsdaten abgestimmt ist und bei unbekannten Eingabedaten keine gute Leistung erbringen wird. (...) Du benötigst etwas Variation im Trainingsdatensatz und passt die Parameter so an, dass das Modell nicht einfach alle Eingabedaten auswendig lernt und darauf basierend die Klassifizierung vornimmt, sondern dass das Modell das Konzept eines Objekts lernen soll. (...) Die richtige Konfiguration für deine Anwendung zu finden, erfordert oft Ausprobieren. (...) (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	

K3	Theorie	Workflow	1. In OpenMV: Datensatz erstellen 2. In OpenMV: Daten nach Edge Impulse hochladen 3. In Edge Impulse: Daten erfassen 4. In Edge Impulse: Einen Impuls kreieren 5. In Edge Impulse: Features erzeugen 6. In Edge Impulse: Das Modell trainieren 7. In Edge Impulse: Das Modell testen 8. In Edge Impulse: Export nach OpenMV 9. In OpenMV: Das Python-Skript ausführen (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	Bis hierher: 10 Min.

		Edge Impulse Account kreieren	In Edge Impulse: Benutzername, E-Mail-Adresse, Passwort (merken !!), Bedingungen zustimmen, E-Mail empfangen, Account aktivieren -> Welcome
			Create New Project, Name eingeben, Personal, Create New Project.
			Target OpenMV H7 Plus auswählen
			Wenn später beim Hochladen der Daten nach Edge Impuls ein Authentifizierungs-Fehler gemeldet wird: Keys/ Add new API key: Development Key Warnung ignorieren
			
			In OpenMV IDE: Extras/ Datensatzeditor/ Neuer Datensatz: Neuen
			Ordner Phi_Psi_Omega erstellen und auswählen
			Bearbeite das ereits geöffnete Bildaufnahmeskript:
			Die Kamera steht auf dem Kopf. Füge deshalb hinter Zeile 12 ein:
			`csi0.vflip(True) # Flips the image vertically`
			`csi0.hmirror(True) # Mirrors the image horizontally`
			Wir **beginnen** mit dem Buchstaben Omega, weil die OpenMV IDE die Ordner alphabetisch geordnet anzeigt. In der Werkzeugleiste: Klicke auf das Symbol Neuer Klassenordner/ letter_Omega erstellen. Führe nun das Bildaufnahmeskript aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst.
			Fokussiere den Buchstaben Omega mit der Kamera und klicke auf das Symbol Daten erfassen, um ein Bild davon aufzunehmen. Wiederhole dies 30x. Ändere Abstand und Winkel zwischen den Aufnahmen.
			Ebenso: In der Werkzeugleiste ... letter_Phi ... Buchstabe Phi
			Ebenso: In der Werkzeugleiste ... letter_Psi ... Buchstabe Psi
			Ebenso: In der Werkzeugleiste ... unknown ... Buchstabe U oder leere Tafel
			(https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	

K3	Aufgabe	Fotos machen		10 Min.

		Fotos zu Edge Impulse übertragen Daten erfassen Impuls kreieren	In **OpenMV IDE:** Extras/ Datensatzeditor/ Export/ In Edge Impulse hochladen/ Projekt wählen/ Trainingsdaten 80% Testdaten 20% -> 120 Dateien hinzugefügt In **Edge Impulse:** Öffne dein Projekt und navigiere zu **„Data Acquisition“**. Du siehst, dass die **Bilder** hochgeladen und entsprechend den von Ihnen erstellten Klassen **gelabelt** wurden. **Die Frage:** "Are you building an object detection project" muss mit NO beantwortet werden. Wenn du das nicht tut, löscht Edge Impuls die Labels. Abhilfe: Du kannst über Project info/ Project info/ Labeling method wieder "One label per data item" einstellen. Mit dem Data Acquisition Tool kannst du die Bildbeispiele durchsuchen und jene entfernen, die du für das Training als nicht wertvoll erachtest (z. B. wenn eines der Bilder zu unscharf ist). Wenn du mit den Datensamples zufrieden sind, kannst du mit der Gestaltung deines Impulses fortfahren. Ein Impuls ist – kurz gesagt – ein Rezept, anhand dessen das Modell trainiert wird. Er definiert Aktionen, die auf Ihre Eingabedaten angewendet werden, um diese für das maschinelle Lernen besser geeignet zu machen, sowie einen Lernblock, der den Algorithmus für die Klassifizierung festlegt. Navigiere im Menü unter **„Impulse Design“** zu **„Create Impulse“** und füge sowohl einen **Image Processing Block** als auch einen **Transfer Learning Block** hinzu. Klicke auf **„Save Impulse“**, um die angepassten Einstellungen zu übernehmen. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	
K3	Aufgabe			10 Min.

K3	Aufgabe	Features erzeugen	In **Edge Impulse:** In diesem Schritt passt du die Bildeinstellungen an und generierst die Merkmale aus den Eingabedaten. Merkmale sind eindeutige Eigenschaften, die vom Klassifizierungsalgorithmus verwendet werden, um die Objekte zu erkennen. (...) Navigieren im Menü unter „Impulse Design“ zum Punkt **„Image“**. Stelle die Farbtiefe auf „RGB“ ein und **speichere die Parameter**. Klicke anschließend auf **„Generate Features“**. Der Analysevorgang wird – abhängig von der Anzahl der hochgeladenen Bilder – einige Zeit in Anspruch nehmen. Sobald er abgeschlossen ist, kannst du die Ergebnisse begutachten. Im Feature Explorer siehst du eine Visualisierung der Merkmale. Du kannst erkennen, dass einige Buchstaben (Punkte verschiedener Farbe) nur schwer voneinander zu unterscheiden sind, weil sich einige Datenpunkte in unmittelbarer Nähe zueinander befinden. Gibt es einen Buchstaben, der sich leichter abgrenzen lässt? (...) (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	5 Min.

K3	Aufgabe	Modell trainieren	In **Edge Impulse:** Nun, da die Merkmale deiner Bilddaten bereitstehen, um für das eigentliche Training verwendet zu werden, kannst du im Menü zum Punkt **„Transfer Learning“** navigieren. Du musst die Einstellungen nun geringfügig anpassen. Legen die „Anzahl der Trainingszyklen“ auf einen Wert fest, der gute Ergebnisse liefert. In diesem Beispiel übernehem wir die Voreinstellung 20. Dies bestimmt, wie oft das Modell trainiert wird. Das Modell verbessert sich mit jedem Zyklus. (...) Wähle den **Model-Typ** MobileNetV2 96x96 0.35 (final layer: 16 neurons, 0.1 dropout) (...) Klicke auf **„Training starten“**, um das Machine-Learning-Modell zu trainieren. Eine kleine Anzahl von Bildern – das Validierungsset – wird beiseitegelegt, bevor das Training beginnt; dies dient der Validierung des trainierten Modells. Dies ist nicht mit dem Testset zu verwechseln, das zur Evaluierung des endgültigen Modells verwendet werden kann. Sobald das Training abgeschlossen ist, erhältst du Statistiken darüber, wie gut das Modell während der Validierung abgeschnitten hat. Idealerweise erreichst du für jedes Objekt eine Genauigkeit von 100 %. Solltest du schlechte Ergebnissez erhalten, enthält dein Datensatz möglicherweise Bilder, die für die zu klassifizierenden Objekte nicht repräsentativ sind und daher entfernt werden sollten. **Training output** zeigt, wie sich **accuracy** und **val_loss** mit jedem Zyklus verändern. Die **Confusion matrix** zeigt die Genauigkeit des ML-Modells nach dem letzten Trainingszyklus an. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	5 Min.

K3	Aufgabe	Modell testen	In **Edge Impulse:** Nach dem Training des Modells hast du eine Vorstellung davon, wie gut das Modell auf den Daten abschneidet, die ihm aus dem Training bekannt sind. Dies ist jedoch nur die halbe Wahrheit. Du musst auch wissen, wie gut es auf bisher ungesehenen Daten performt. In fast jeder realen Anwendung wird ein Modell ausschließlich mit ungesehenen Daten konfrontiert. Die Fähigkeit, damit umzugehen, ist von entscheidender Bedeutung. Das Edge Impulse Studio stellt ein Tool bereit, mit dem du das Modell auf einfache Weise testen kannst. Du findest es unter dem Menüpunkt **„Model Testing“**. Die Ergebnisse des Modelltests geben dir Aufschluss über die Leistungsfähigkeit. Erzielt das Modell während des Tests schlechte Ergebnisse, wies jedoch nach dem Training eine hohe Genauigkeit auf, könnte dies auf ein Overfitting hindeuten. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	5 Min.
K4	Thema	KI-Modell auf OpenMV Cam testen		
K4	Praxis	ML-Modell in Edge Impulse besprechen	In Edge Impulse: ML-Modell öffnen Create Impulse mit den 4 Blöcken Image mit den Processed Features Transfer Learning mit der Confusion Matrix und dem Data Explorer für beide KI-Modelle Model testing für das float32 Modell	5 Min.
K4	Aufgabe	Build OpenMV library Download der Modelle	In **Edge Impulse:** Das ML-Modell ist trainiert und bereits für den Einsatz auf Mikrocontrollern optimiert. Dies erfolgt automatisch im Hintergrund durch Quantisierung. Dabei handelt es sich um einen Prozess, bei dem der Wertebereich der Zahlen innerhalb der Machine-Learning-Modelle eingeschränkt wird, um die Leistung zu steigern – unter Inkaufnahme eines geringfügigen Genauigkeitsverlusts. Wechsle im Menü zum Bereich **„Deployment“**, wähle unter **„Configure your deployment“** die Option **„OpenMV library“** aus und klicke auf **„Build“**. Dadurch wird eine **Bibliothek** erstellt, die deinen Impuls auf der OpenMV Cam ausführt. Entpacke die Datei, sobald der Download abgeschlossen ist. Der Download sollte automatisch starten. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification Die ZIP-Datei enthält nur das int8-Modell. So erhältst du das float32-Modell: Navigiere zu **Dashboard/ Download block output.** Klicke auf das Download-Symbol neben TensorFlow Lite (float32), um den Download zu starten.	10 Min.

K4	Aufgabe	Entpacken Umbenennen Editieren	Im Windows Datei-Explorer:
			Entpacke die Datei ei-<user>-<project>-openmv-v-<number>-impulse-#1.zip
			Benenne die Dateien um:
			Vorher: trained.tflite Nachher: trained_phi_psi_omega_int8.tflite
			Vorher: labels.txt Nachher: labels_phi_psi_omega.txt
			Vorher: ei_image_classification.py
			Nacher: ei_image_classification_phi_psi_omega_int8.py
			Vorher: ei-<user>-<project>tensorflow-lite-float32-model.8.lite
			Nacher: trained_phi_psi_omega_float32.lite
			
			Mit einem Editor:
			Editiere die Datei ei_image_classification_phi_psi_omega_int8.py
			Hinter Zeile 12 einfügen:
			`sensor.set_vflip(True)  # Flips the image vertically`
			`sensor.set_hmirror(True)  # Mirrors the image horizontally`
			Ersetze alle Vorkommen von labels.txt durch: labels_phi_psi_omega.txt
			Ersetze alle Vorkommen von trained.tflite durch: trained_phi_psi_omega_int8.tflite
			Speichern nicht vergessen !!
			Kopiere die geänderte Datei ei_image_classification_phi_psi_omega_int8.py nach:
			ei_image_classification_phi_psi_omega_float32.py
			Editiere die Datei ei_image_classification_phi_psi_omega_float32.py
			Ersetze alle Vorkommen von trained_phi_psi_omega_int8.tflite durch:
			trained_phi_psi_omega_float32.lite
				10 Min.

K4	Aufgabe	Anschließen Kopieren Testen	Im Windows Datei-Explorer: Schließe die Kamera an den Computer an. Das integrierte USB-Laufwerk wird als OpenMV D: angezeigt, Kopiere die Dateien vom Computer auf Laufwerk D: ei_image_classification_phi_psi_omega_float32.py ei_image_classification_phi_psi_omega_int8.py labels_phi_psi_omega.txt trained_phi_psi_omega_int8.tflite trained_phi_psi_omega_float32.lite In OpenMV IDE testen: Öffne die Datei: ei_image_classification_phi_psi_omega_int8.py Führe nun das KI-Modell aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst. Fokussiere nun den Buchstaben Phi mit der Kamera und beobachte die Ausgaben am Seriellen Terminal. Ändere Abstand und Winkel zum Buchstaben Phi. Dieses KI-Modell läuft mit ca. 13 Frames per Second. Stoppe das Skript und beobachte die Ausgabe. Im KI-Modell wird softmax für Mehrklassenvorhersagen verwendet. Die Summe aller von softmax erzeugten Ausgaben ist 1. Mit dem Speichern-Symbol speicherst du die Ausgaben in einer Log-Datei auf deinem Computer. Mit dem Wischen-Symbol löschst du die Ausgaben im Seriellen Terminal bevor du das KI-Model erneut ausführt. Wiederhole den Test mit der Datei ei_image_classification_phi_psi_omega_float32.py Dieses (große) Modell läuft mit ca. 3 Frames per Second.	15 Min.
K4	Aufgabe	https://netron.app/	In Netron.app: Öffne die Datei: trained_phi_psi_omega_int8.tflite Input und Output anschauen, int8, Dimensionen, Softmax Öffne die Datei: trained_phi_psi_omega_float32.lite Input und Output anschauen, float32, Dimensionen, Softmax	5 Min.
K5	Thema	KI-Modell verbessern		
K6	Thema	ESP32 Ansteuerung		

K6	Theorie	Versorgungsspannung und digitale Pegel	Die **OpenMV Cam H7 Plus** wird mit **3.6 – 5 V** versorgt. Die **Versorgungspannung** wird an die Pins VIN und GND angeschlossen. Wenn sowohl USB als auch VIN/LiPo anliegen, hat der VIN/LiPo-Eingang Vorrang – der integrierte Stromwahlschalter wählt diesen gegenüber USB aus, um das Board mit Strom zu versorgen. Die OpenMV Cam H7 Plus hat 10 I/O-Pins, 5-V-tolerant mit 3,3-V-Ausgang, (...) P6 ist im ADC- oder DAC-Modus nicht 5-V-tolerant. Andere OpenMV Cams Einige OpenMV Cams verfügen über 5-V-tolerante I/O-Pins (ein 5-V-Signal kann ohne Beschädigung direkt angelegt werden); **andere** betreiben ihre I/O-Schnittstellen mit 3,3 V oder 1,8 V und benötigen für Signale mit höherer Spannung einen Pegelwandler. Der Anschluss einer 5-V-Quelle an einen nicht toleranten Pin kann den Chip beschädigen. (https://docs.openmv.io/openmvcam/tutorial) Die **ESP32-Platine** wird über USB mit **5 V versorgt**. Die ESP32-Platine verfügt über digitale Ein-/Ausgangs-Pins (GPIO Pins - General Purpose Input/Output Pins). Diese digitalen Eingänge/Ausgänge arbeiten mit 3,3V. Es darf keine 5V Spannung an die ESP32 Chip Pins angeschlossen werden! Die Pins sind nicht 5V-tolerant, das Anlegen von mehr als 3,3V an einem Pin wird den Chip beschädigen. (AZ-Delivery: ESP-32 Dev Kit C V4_DE.pdf) **Schlussfolgerung**: Wir können die OpenMV Cam direkt an den ESP32 anschließen.	5 Min.
K6	Aufgabe	Verdrahtung der UART-Kommunikation zwischen OpenMV Cam und ESP32	OpenMV Cam und ESP32-Platine von der **Spannungsversorgung trennen !!** Verbinde den OpenMV GND Pin mit dem ESP32 GND Pin. Verbinde den OpenMV Cam UART3 TX Pin (P4) mit dem ESP32 RX2 Pin (GPIO16) Verbinde den OpenMV Cam UART3 RX Pin (P5) mit dem ESP32 TX2 Pin (GPIO17) **Begründung:** Auf der OpenMV Cam arbeiten wir mit UART3. Auf der ESP32-Platine arbeiten wir mit UART2.	5 Min.

K6	Aufgabe	Ausführung des Programms ei_image_classification_phi_psi_omega_int8_serial_output.py	Schließe die OpenMV Cam über USB an den Computer an. Starte die OpenMV IDE. Öffne das Programm ei_image_classification_phi_psi_omega_int8_serial_output.py. Das Programm ist eine Erweiterung des Programms, mit dem wir in den letzten beiden Stunden gearbeitet haben. Das Programm macht eine Ausgabe, wenn Softmax für ein Label einen Wert größer als den Schwellwert 0,5 liefert. In der Ausgabe steht das erkannte Label und die laufende Nummer des Schnappschusses. Nur nach jedem 10ten Schnappschuss wird eine Ausgabe gemacht, damit wir sie gut beobachten können. Die Strings werden durch Leerzeichen voneinander getrennt. Das ist notwendig, damit der Empfänger Anfang und Ende der Strings erkennen kann. print() schreibt die Ausgabe in das Serielle Terminal. uart.write() schreibt die Ausgabe auf den UART3. Während das Programm läuft, leuchtet die LED blau. Der Schwellwert kann geändert werden. Ebenso kann geändert werden, nach wie vielen Schnappschüssen eine Ausgabe gemacht wird. Führe nun das Programm aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst. Fokussiere nun den Buchstaben Phi mit der Kamera und beobachte die Ausgaben am Seriellen Terminal. Bewege die Tafel mit dem Buchstaben hin und her und beobachte die Ausgabe im Seriellen Terminal.	

K6	Aufgabe	Ausführung des Programms ESP32_UART_control_input_only.py	Schließe die ESP32-Platine über den zweiten USB-Anschluss an den Computer an. Starte die **Thonny IDE**. Öffne das Programm **ESP32_UART_control_input_only.py.** **uart.read()** empfängt ein **Bytes Object** über UART2. Wenn die empfangene Nachricht **nicht None** ist, versucht das Programm die Folge von Bytes zu dekodieren. Die **Exception** fängt den Unicode Error ab, der beim Stopp des OpenMV Cam Programm entsteht. sleep(5) sorgt dafür, dass das Byte Objekt nur alle 5 Sekunden gelesen wird. Dieser Wert kann **geändert** werden. Es simuliert, dass der ESP32 langsamer liest als die OpenMV Cam Ausgaben macht. Durch das langsamere Lesen enthält das Bytes Objekt eine Reihe von Labeln und Nummern. Nun soll das Programm nur das letzte Label und die letzte Nummer auswerten. Dazu spalten wir das Bytes Objekt in eine Liste mit Labeln und Nummer auf. Danach können wir die beiden letzten Elemente der Liste zusammenbauen und dekodieren. Führe nun das Programm aus und beobachte die Ausgaben in der ESP32 Shell. Bewege die Tafel mit dem Buchstaben hin und her und vergewissere dich, dass das letzte Label angezeigt wird.	Bis hierher: 10 Min.

K6	Aufgabe	OpenMV Cam ohne IDE betreiben	In einer Anwendung wird die OpenMV Cam ohne IDE betrieben. Dupliziere das Programm ei_image_classification_phi_psi_omega_int8_serial_output.py mit dem Windows Datei-Explorer auf dem USB-Laufwerk der OpenMV Cam. Es wir als OpenMV D: angezeigt, Lösche die Originaldatei main.py oder benenne sie um. Benenne das Duplikat in main.py um. Trenne die OpenMV Cam vom Computer. Verbinde den OpenMV VIN Pin mit dem ESP32 5V Pin der ESP32-Platine. Nun bootet die OpenMV Cam und schreibt die Ausgaben auf den UART3. Während das Programm läuft, leuchtet die LED blau. Beobachte die Ausgaben in der ESP32 Shell.	10 Min.
K6	Aufgabe	Verdrahtung der parallelen Kommunikation zwischen OpenMV Cam und ESP32	Die schnellste Kommunikation geht über den parallelen Output der OpenMV Cam. Mit 2 Pins kannst du 4 Zustände unterscheiden. Das ist ausreichend für die 4 Labels unseres KI-Modells. OpenMV Cam und ESP32-Platine von der **Spannungsversorgung trennen !!** Verbinde den OpenMV GND Pin mit dem ESP32 GND Pin. Verbinde den OpenMV Cam Pin P0 (PB15) mit dem ESP32 Pin 18 (GPIO18) Verbinde den OpenMV Cam Pin P1 (PB14) mit dem ESP32 Pin 19 (GPIO19)	5 Min.

K6	Aufgabe	Ausführung des Programms ei_image_classification_phi_psi_omega_int8_parallel_output.py	Schließe die OpenMV Cam über USB an den Computer an. Starte die OpenMV IDE. Öffne das Programm ei_image_classification_phi_psi_omega_int8_parallel_output.py. Das Programm ist eine Erweiterung des Programms, mit dem wir in den letzten beiden Stunden gearbeitet haben. Das Programm macht eine Ausgabe, wenn Softmax für ein Label einen Wert größer als den Schwellwert 0,5 liefert. In der Ausgabe steht das erkannte Label und die laufende Nummer des Schnappschusses. Nur nach jedem 10ten Schnappschuss wird eine Ausgabe gemacht, damit wir sie gut beobachten können. print() schreibt die Ausgabe in das Serielle Terminal. Das Programm macht aus dem Label String einen 2 Bit Code, der als Status der parallelen Output Pins ausgegeben wird. Während das Programm läuft, leuchtet die LED blau. Der Schwellwert kann geändert werden. Ebenso kann geändert werden, nach wie vielen Schnappschüssen eine Ausgabe gemacht wird. Führe nun das Programm aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst. Fokussiere nun den Buchstaben S mit der Kamera und beobachte die Ausgaben am Seriellen Terminal. Bewege die Tafel mit dem Buchstaben hin und her und beobachte die Ausgabe im Seriellen Terminal.	

K6	Aufgabe	Ausführung des Programms ESP32_parallel_input.py	Schließe die ESP32-Platine über den zweiten USB-Anschluss an den Computer an. Starte die **Thonny IDE.** Öffne das Programm **ESP32_parallel_input.py.** Die **parallelen Input Pins des ESP32** bekommen den **Status** der parallelen Output Pins der OpenMV Cam. Aus beiden Bits wird ein Label Code gemacht. Anschließend wird der Label Code dekodiert. sleep(5) sorgt dafür, dass der Status nur alle 5 Sekunden gelesen wird. Dieser Wert kann geändert werden. Es simuliert, dass der ESP32 langsamer liest als die OpenMV Cam Ausgaben macht. Anders als bei der seriellen Kommunikation gibt es bei der parallelen Kommunikation keinen "alten" Status. Der ESP32 liest immer den aktuellen Status. Führe nun das Programm aus und beobachte die Ausgaben in der ESP32 Shell. Bewege die Tafel mit dem Buchstaben hin und her und vergewissere dich, dass das akuelle Label angezeigt wird.	Bis hierher 10 Min.
K7	Thema	Kreis erkennen		